

Pontifícia Universidade Católica de Minas Gerais

Disciplina: Teste de Software

Teste de Mutação com StrykerJS: Avaliando a Qualidade de uma Suíte de Testes

Aluno: Vitor Costa Vianna

Matrícula: 857863

Professor: Cleiton Tavares

Repositório: github.com/VitorCostaVianna/operacoes-mutante

1. Análise Inicial

O projeto `operacoes-mutante` tem 50 funções aritméticas em `src/operacoes.js` e uma suíte de 50 testes Jest, todos passando. A cobertura foi medida com `npm test -- --coverage`.

Métrica	Valor inicial
Cobertura de linhas	98,64%
Cobertura de funções	100%
Cobertura de instruções (statements)	85,41%
Cobertura de desvios (branches)	58,82%
Pontuação de mutação (213 mutantes)	73,71%

157										44		12	
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total	
	Of total	Of covered											
JS operacoes.js	73.71	78.11	154	44	3	12	0	0	0	157	56	213	

Figura 1 – Primeira execução do Stryker: 157 mutantes detectados (154 mortos + 3 timeout), 44 sobreviventes e 12 sem cobertura.

Análise inicial: pela cobertura, a suíte parece quase completa, já que todas as funções são chamadas e só uma linha não é executada. Mas cobertura mede apenas se o código foi executado, não se o teste verifica o resultado. O Stryker mostra a diferença: 44 mutantes sobreviveram em código coberto. A linha executou, o comportamento mudou e nenhuma asserção percebeu. A causa está no estilo da suíte: cada teste faz uma única asserção, com uma única entrada do caminho feliz. Funções booleanas só são testadas com o caso `true`, nenhum teste usa valores de fronteira ou entradas inválidas (arrays vazios, negativos, zero), e algumas entradas escolhidas anulam justamente o trecho que deveria ser verificado. A cobertura de desvios (58,82%) já indicava isso, mas só o teste de mutação mostra onde e por que a suíte falha.

2. Análise de Mutantes Críticos

2.1 `isMaiorQue`: troca de `>` por `>=` (EqualityOperator, linha 104)

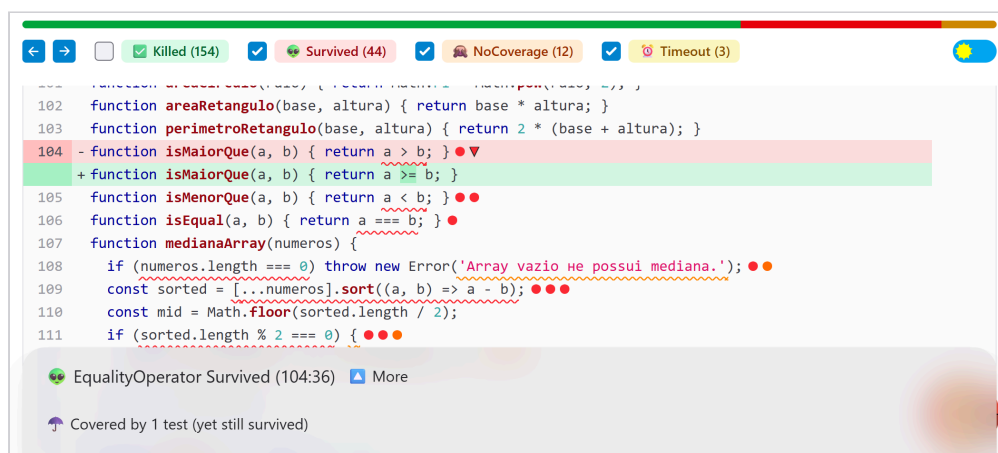
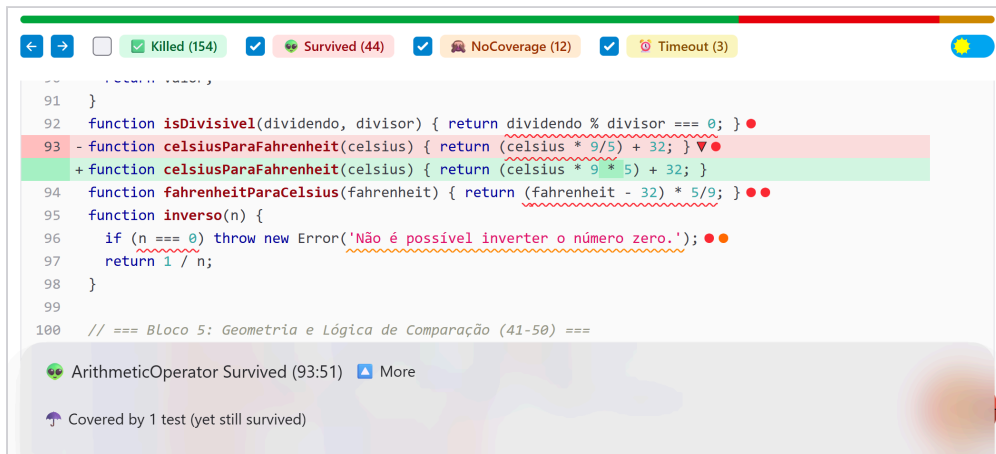


Figura 2 – Mutante sobrevivente em `isMaiorQue`: `a > b` virou `a >= b`.

O que a mutação fez: passou a considerar dois valores iguais como "maior", um erro clássico de fronteira (off-by-one). Por que sobreviveu: o único teste é `expect(isMaiorQue(10, 5)).toBe(true)`. Com 10 e 5, `10 > 5` e `10 >= 5` são ambos `true`. Os dois operadores só divergem quando `a === b`, e esse caso nunca é testado. Na mesma linha também sobreviveu o mutante `return true`, porque nenhum teste espera `false`.

2.2 celsiusParaFahrenheit: troca de 9/5 por 9*5 (ArithmeticOperator, linha 93)



```
91 }
92 function isDivisivel(dividendo, divisor) { return dividendo % divisor === 0; }
93 - function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
+ function celsiusParaFahrenheit(celsius) { return (celsius * 9 * 5) + 32; }
94 function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5/9; }
95 function inverso(n) {
96   if (n === 0) throw new Error('Não é possível inverter o número zero. ');
97   return 1 / n;
98 }
99
100 // === Bloco 5: Geometria e Lógica de Comparação (41-50) ===
```

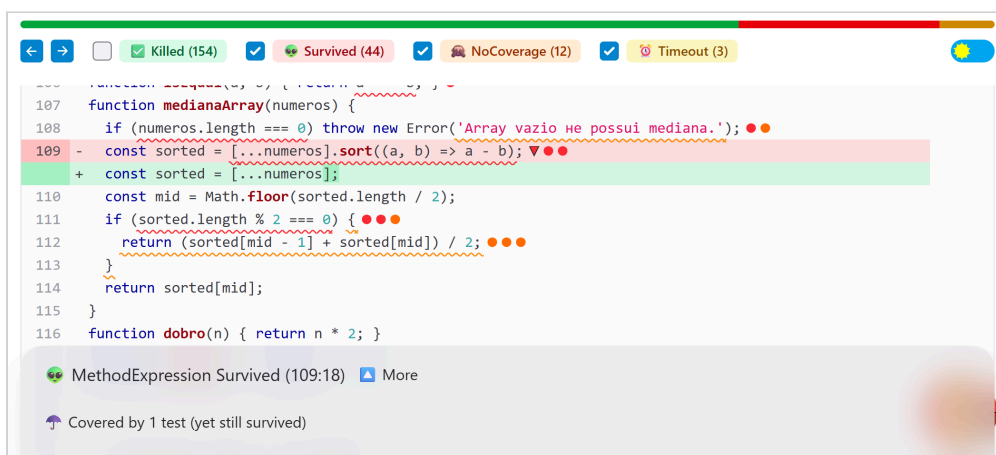
ArithmeticOperator Survived (93:51) More

Covered by 1 test (yet still survived)

Figura 3 – Mutante sobrevivente em `celsiusParaFahrenheit`: o fator de conversão 9/5 virou 9*5.

O que a mutação fez: trocou o fator de conversão, e a fórmula passou a dar resultados completamente errados (100 °C viraria 4532 °F). Por que sobreviveu: o teste usa `celsiusParaFahrenheit(0)`. Zero multiplicado ou dividido por qualquer valor continua zero, então original e mutante retornam `0 + 32 = 32`. A entrada escolhida anula exatamente a parte mutada. O teste executa a fórmula, mas é incapaz de distinguir uma fórmula certa de uma errada. O mesmo acontece em `fahrenheitParaCelsius(32)`, onde `(32 - 32) = 0`.

2.3 medianaArray: remoção da ordenação (MethodExpression, linha 109)



```
107 function medianaArray(numeros) {
108   if (numeros.length === 0) throw new Error('Array vazio He possui mediana. ');
109 - const sorted = [...numeros].sort((a, b) => a - b);
+ const sorted = [...numeros];
110   const mid = Math.floor(sorted.length / 2);
111   if (sorted.length % 2 === 0) {
112     return (sorted[mid - 1] + sorted[mid]) / 2;
113   }
114   return sorted[mid];
115 }
116 function dobro(n) { return n * 2; }
```

MethodExpression Survived (109:18) More

Covered by 1 test (yet still survived)

Figura 4 – Mutante sobrevivente em `medianaArray`: a chamada a `.sort()` foi removida.

O que a mutação fez: removeu a ordenação, sem a qual a mediana não faz sentido. Por que sobreviveu: o teste usa `[1, 2, 3, 4, 5]`, um array já ordenado, então ordenar ou não dá o mesmo resultado. Por isso também sobreviveram os mutantes que trocam o comparador (`a + b` e `() => undefined`). E, como o array tem tamanho ímpar, o ramo que calcula a mediana de arrays pares (linhas 111 e 112) nunca é exercitado.

3. Solução Implementada

Os 50 testes originais foram mantidos, pois não estão errados, apenas incompletos. Os novos testes foram adicionados num bloco separado, `describe('Testes adicionais para matar mutantes')`. Para os mutantes analisados acima:

```
// 2.1 - caso false e fronteira a === b
test('isMaiorQue retorna false quando a é menor ou igual a b', () => {
  expect(isMaiorQue(5, 10)).toBe(false); // mata "return true"
  expect(isMaiorQue(5, 5)).toBe(false); // mata "a >= b"
});

// 2.2 - entradas que não zeram a fórmula
test('converte 100 °C para 212 °F', () => { expect(celsiusParaFahrenheit(100)).toBe(212); });
test('converte 212 °F para 100 °C', () => { expect(fahrenheitParaCelsius(212)).toBe(100); });

// 2.3 - array desordenado e array de tamanho par
test('medianaArray ordena o array antes de calcular', () => {
  expect(medianaArray([5, 1, 3])).toBe(3); // sem ordenação retorna 1
});
test('medianaArray de array par é a média dos dois centrais', () => {
  expect(medianaArray([4, 1, 3, 2])).toBe(2.5); // exercita as linhas 111 e 112
});
```

Por que são eficazes: cada entrada foi escolhida para que o código original e o mutante produzam resultados diferentes. `isMaiorQue(5, 5)` é exatamente o ponto onde `>` e `>=` divergem. Com 100, os mutantes da conversão retornam 4532 e $\approx 34,2$ em vez de 212. Com `[5, 1, 3]`, a versão sem ordenação retorna o elemento do meio da posição original (1) em vez da mediana (3). A mesma lógica foi aplicada aos demais sobreviventes, totalizando 25 novos testes: casos que devem retornar `false` (`isPar`, `isImpar`, `isDivisivel`, `isMenorQue`, `isEqual`, `isPrimo(9)`), fronteiras (`raizQuadrada(0)`, `fatorial(0)`, `isPrimo(1)`), valores fora do intervalo no `clamp`, entradas inválidas (arrays vazios, negativos, `inverso(0)`) e verificação do texto das mensagens de erro com `toThrow('mensagem')`, já que `toThrow()` sem argumento não detectava a mensagem trocada por `""`.

Mutantes equivalentes: 7 sobreviventes não podem ser mortos por nenhum teste, porque não alteram o comportamento. O atalho `if (n === 0 || n === 1) return 1` do `fatorial` é redundante (sem ele o laço não roda e o resultado continua 1), o `reduce` com valor inicial 1 já trata o array vazio no `produtoArray`, e no `clamp`, com valor `=== min`, retornar `min` ou valor é o mesmo. Esses casos foram marcados com `// Stryker disable next-line` e a justificativa, sem alterar a lógica do código.

4. Resultados Finais

Os 75 testes passam com o código original (`npm test`), e o Stryker foi executado novamente:

	Inicial	Final
Testes	50	75
Mutantes detectados (mortos + timeout)	157	202
Sobreviventes	44	0
Sem cobertura	12	0
Ignorados (equivalentes)	0	11
Pontuação de mutação	73,71%	100%

202												
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
JS operacoes.js	 100.00	 100.00	199	0	3	0	11	0	0	202	0	213

Figura 5 – Execução final do Stryker: 202 mutantes detectados, nenhum sobrevivente e pontuação de mutação de 100%.

A pontuação passou de 73,71% para 100%, acima da meta de 98%. Os 11 ignorados incluem os 7 equivalentes e mais 4 mutantes das mesmas linhas, porque o comentário de disable vale para o tipo de mutação na linha inteira. Sem nenhum disable, a pontuação seria de $206/213 = 96,7\%$: todos os mutantes que podem ser mortos foram mortos.

5. Conclusão

Este trabalho mostrou na prática que cobertura de código alta não garante uma suíte de testes eficaz. Com 98,64% das linhas executadas, a suíte original deixava passar 1 em cada 4 defeitos introduzidos: comparações erradas, fórmulas trocadas e validações removidas. A cobertura responde "o código foi executado?", enquanto o teste de mutação responde "se o código estiver errado, os testes percebem?", que é a pergunta que importa. Além do número, o relatório do Stryker funciona como guia: cada sobrevivente aponta um caso concreto que faltava testar (fronteiras, casos negativos, entradas inválidas e entradas que distinguem o certo do errado). A análise dos sobreviventes também exige julgamento, porque nem todo mutante indica falha (os equivalentes não podem ser mortos). Por ser mais custoso que a cobertura, o teste de mutação é mais útil como avaliação periódica da qualidade da suíte, e deixa os testes de fato capazes de proteger o sistema contra regressões.